Sistemas Operativos Avanzados - Primer parcial

Merkel, Germán - D.N.I. 40.853.806

Grupo 15

Punto 1

> En los STR dadas las características de oportunidad y diseño de carga: comente como trabajan ambas en conjunto.

Los sistemas de tiempo real deben producir valores correctos, al igual que los sistemas de propósito general, pero deben hacerlo en un tiempo determinado (antes de las deadlines), este concepto se lo conoce como "oportunidad". Además, estos sistemas deben estar diseñados de tal manera que no colapsen cuando se encuentran en condiciones de carga máxima, este concepto se lo conoce como "diseño de carga".

Ambos conceptos se relacionan dado que el sistema debe ser diseñado teniendo en cuenta que en una condición de carga máxima debe seguir ofreciendo resultados correctos en el tiempo adecuado.

Punto 2

> ¿Qué tipo de planificador es el algoritmo de prioridad monótona en tasa?

El algoritmo de prioridad monótona en tasa es un algoritmo de planificación **basado en prioridades**, **preemptive**, con control de admisión y **estático**. Al ser un algoritmo de planificación estático, se trata de un algoritmo cuyos parámetros son fijos, esto es, las políticas de prioridades que utiliza no varían. Al ser preemptive, significa que el planificador desaloja a los procesos cuando un nuevo proceso con mayor prioridad aparece en la cola de listos.

Se dice también que es un algoritmo **óptimo** dado que si un conjunto de tareas no puede ser planificada por este algoritmo, entonces no podrá ser planificado por ningún otro algoritmo de planificación con *prioridades estáticas*.

Punto 3

> ¿En qué consiste el concepto de compilación cruzada?¿Cuál es la relación que existe entre este concepto con Arduino UNO?

La compilación cruzada es una técnica que permite realizar una compilación desde una arquitectura X hacia una arquitectura Y, esto es, crear un binario desde una arquitectura que será ejecutada en otra arquitectura diferente. Por ejemplo, desde mi PC de escritorio puedo compilar un programa que luego correrá dentro de un sistema embebido que utilice una arquitectura distinta.

La compilación cruzada se relaciona con la placa Arduino UNO dado que se realiza una compilación tal cual fue descripta en el párrafo anterior. Normalmente se realiza la compilación cruzada desde una computadora de escritorio, que compila para la arquitectura que utiliza el microcontrolador Atmega de Arduino UNO, y luego este programa compilado "se quema" en la ROM del Arduino.

Punto 4

> ¿Por qué es importante el uso de máquina de estados en SE?

Las maquinas de estados ofrecen un modelo gráfico y matemático de representar los estados y eventos de un sistema. Esta representación facilita el diseño y la construcción del sistema embebido, el cual está fuertemente relacionado con los conceptos de estados y eventos, dado que vimos en el trabajo práctico que la mayor parte del software es dedicado a recibir eventos externos y reaccionar ante ellos cambiando el estado actual del sistema.

Al diseñar la máquina de estados utilizando la representación gráfica, es sencillo crear el diseño del sistema y realizar su validación. En el momento de la construcción, se puede extraer la *lookup table* de la representación matricial del autómata finito, y por lo tanto, gran parte de la lógica del manejo de estados se ve facilitada.

Punto 5

> Indique si la siguiente oración es correcta, justifique para que sea considerada como válida. "La forma de depurar (debuggear) un programa en Arduino es a través de GDB"

La oración es falsa. La forma de depurar un programa en Arduino es utilizando depuración indirecta, esto es, realizando "prints" en un puerto que será recibido por la PC a la cual esté conectado. Esta forma de depuración se encuentra incluida en el código fuente, y una buena práctica es hacer que el preprocesador pueda desactivar esta porción de código.

GDB se puede utilizar en sistemas embebidos programados, por ejemplo, en Raspberry Pi, ya sea utilizando un cliente servidor, o usando GDB dentro del

Raspberry.

Punto 6

> ¿Cuál es la diferencia de DFA y NDFA? ¿Cuál usó en su trabajo práctico?

La diferencia puede verse en el nombre de ambos tipos de autómatas: DFA signfica Deterministic Finite Automata (o autómata finito determinístico) y NDFA significa Non-Deterministic Finite Automata (o autómata finito no determinístico). Se dice que DFA es determinístico dado que conocemos qué estado será el siguiente para cada evento lanzado. Mientras que se dice que NDFA es no determinístico dado que para un determinado estado y un determinado evento, existe más de un estado final, esto es, para un estado A, un evento E puede cambiar el estado hacia el estado B o hacia el estado C.

En el trabajo práctico GOOD FUN se ha utilizado un autómata finito dado que para un estado y un evento es posible saber cual será la transición a realizar.

Punto 7

> ¿Qué agregado debería realizar el circuito de Tinkercad del Trabajo Práctico para que dicho proyecto sea Internet de las cosas?

Para que un sistema sea considerado un sistema de IoT se requieren las siguientes condiciones:

- Transmitir datos a Internet
- a través de un **gateway**
- y los datos deben ser procesados en la nube.

Por lo tanto, si se quisiese que la cabina de fotos GOOD FUN sea considerado un sistema IoT se podría agregar una placa de red al sistema, transmitir datos a través de Internet como el datetime de entrada del humano, salida del gas, detención de salida de gas, y salida del humano. Luego de transmitir estos datos, en la capa Cloud de la arquitectura IoT se podría realizar el procesamiento de estos datos para obtener estadísticas de uso.

Punto 8

> ¿Qué ventajas tiene utilizar la función `delay()` por sobre los temporizadores? Entonces, ¿por qué no fue permitió utilizarla en el TP1?

La principal ventaja de la función `delay()` por sobre los temporizadores (ya sea por software, hardware o híbridos) es la facilidad de desarrollo: se trata de

incluir la función `delay()` en el código, indicando el tiempo en milisegundos, y el sistema se bloqueará y luego de ese tiempo se ejecutará la instrucción siguiente.

Sin embargo, la función `delay()` tiene un problema: como se ha dicho, es una llamada bloqueante. Mientras dicha función ejecuta, el sistema no podrá detectar los eventos, y por lo tanto, probablemente se pierdan cambios de estados que debían hacerse. Es por esta razón que no se permitió utilizarla en el TP 1.

Punto 9

> ¿Qué cambios son requeridos en el TP1, si Cerebro solicita agregar un LED que cambie su intensidad al producirse un cambio de estado del FSM?

Para variar la intensidad del LED, es necesario realizar PWM con Arduino en el pin del LED. En el caso del trabajo práctico GOOD FUN, deberíamos agregar un LED conectado en un pin digital del Arduino, luego se debería modificar la máquina de estados para que con determinado evento realice un `analogWrite` sobre el pin del LED, indicando un número del 0 al 255, donde 0 representa la intensidad de brillo más baja (apagado) y 255 representa la intensidad de brillo más alta.

La modificación de la máquina de estados puede requerir que se agregue un nuevo puntero a función en la *lookup table* de la máquina de estados, o modificar una función ya existente.

Punto 10

> ¿Por qué se le pide en el TP 1 que respete la nomenclatura de colores en los conectores de la fuente de energía? ¿Qué sucede si no se respetan los colores sugeridos?

Se pide que se respeten los colores para los cables positivos (rojos) y negativos (negros), y que se utilicen distintos colores para los cables conectados a los pines de Arduino dado que se trata de un estándar que facilita el trabajo en equipo. Si se respetan los colores de los cables, otro participante del grupo (o mismo los profesores) puede identificar rápidamente qué cables son aquellos que están conectados a la fuente de energía, cuáles son positivos y negativos, qué cables están conectados al Arduino, y diferenciar los cables que salen de un componente.

Si no se respeta la nomenclatura, el sistema seguirá funcionando de todos modos, pero probablemente otra persona que revise el sistema confunda los cables.